Prova Pratica 046 - 13 gennaio 2020

Per aiutare chi trova difficoltà a superare la prova pratica, ho previsto due tipi di prove:

A. **una prova COMPLICATA**, che vi permette di raggiungere un **voto massimo** (nella prova pratica stessa) di **30Lode** ,
B. ed **una prova SEMPLICE**, un po' **meno complicata**, che però vi permette di raggiungere un **voto massimo di 24** perché l'esercizio di programmazione concorrente é meno difficile.

Scegliete voi quale prova svolgere in funzione della vostra preparazione e del grado di angoscia e panico che vi sommerge.

La prova COMPLICATA è composta dagli esercizi **133 e 134**,
La prova SEMPLICE è composta dagli esercizi **132e 134**.
Come vedere l'esercizio 134 è comune alle due prove.

Svolgete SOLO gli esercizi della prova che vi interessa. Se consegnate esercizi di entrambe le prove io NON li correggo !
i file da consegnare **devono** essere collocati nella directory **CONSEGNA** dentro la home directory dell'utente studente.

Esercizio Esame Pratica **- 132 - semaforo semplice**

La circonvallazione di Passogatto (RA) è un anello a senso unico che passa davanti all'osteria. Le auto girano in continuazione nell'anello, sempre e solo in senso orario. In corrispondenza dell'osteria c'è un largo attraversamento pedonale protetto da un semaforo per le auto, che serve a interrompere il flusso di auto per l'unico senso di marcia.

La luce verde ordina alle auto di passare, la luce rossa di fermarsi, la luce arancione di fermarsi o di liberare le strisce pedonali se si sta già passando.

Ogni auto è caratterizzata da un indice univoco compreso tra 0 e il (numero delle auto meno 1).

Le auto possono passare sull'attraversamento pedonale a bassa velocità, impiegando 2 secondi.

Le auto possono cominciare a passare sull'attraversamento pedonale senza aspettare che l'auto precedente sia uscita, ma l'ordine di uscita delle auto deve essere lo stesso dell'entrata.

Oltrepassato l'attraversamento pedonale, ogni auto impiega altri (**5+ suo indice**) secondi a fare tutto il resto del giro per ripresentarsi davanti all'osteria.

Circa ogni **4** secondi, il semaforo comincia la procedura per interrompere il flusso di auto.

Prima mette la luce arancione, poi aspetta **1** secondo, poi aspetta che la zona pedonale sia libera da auto, infine mette la luce rossa.

Dopo **3** secondi di luce rossa, il semaforo rimette la luce verde e le auto ricominciano a passare.

Ci sono 6 auto (numero auto == 6).

Modellare ed implementare il sistema descritto, **utilizzando dei thread POSIX** per **ciascuna auto** e per **il semaforo**, e avvalendosi di mutex e condition variable per la sincronizzazione. Scrivere il Makefile per compilare e linkare i sorgenti. Inserire il necessario controllo di errore. In caso di errore grave, terminare il programma producendo un avviso a video.

Consiglio: Non chiamate auto la vostre funzioni perché esiste una parola chiave del linguaggio C che si chiama così . Meglio chiamare la vostra funzione Auto (con la A maiuscola) .

Esercizio Esame Pratica **- 133 - semaforo complicato**

La circonvallazione di Passogatto (RA) è un anello a senso unico che passa davanti all'osteria. Le auto procedono sempre e solo in senso orario. Davanti all'osteria c'è un largo attraversamento pedonale protetto da un semaforo per le auto, che serve a interrompere il flusso di auto per l'unico senso di marcia. La luce verde ordina alle auto di passare, la luce rossa di fermarsi, la luce arancione di fermarsi o di liberare le strisce pedonali se si sta già passando.

Ogni 1 secondi il main del programma genera 3 auto, ciascuna caratterizzata da un indice compreso tra 0 e 2, che si presentano all'inizio dell'attraversamento pedonale.

Le auto possono passare sull'attraversamento pedonale a bassa velocità, impiegando 2 secondi.

Le auto possono cominciare a passare sull'attraversamento pedonale senza aspettare che l'auto precedente sia uscita, ma l'ordine di uscita delle auto deve essere lo stesso dell'entrata.

Oltrepassato l'attraversamento pedonale, ogni auto impiega altri (3**+ suo indice**) secondi a fare tutto il resto del giro e poi **termina il proprio lavoro e viene eliminata**.

Circa ogni **4** secondi, il semaforo comincia la procedura per interrompere il flusso di auto.

Prima mette la luce arancione, poi aspetta **1** secondo, poi aspetta che la zona pedonale sia libera da auto, infine mette la luce rossa.

Dopo **3** secondi di luce rossa, il semaforo rimette la luce verde e le auto ricominciano a passare.

Modellare ed implementare il sistema descritto, **usando un PROCESSO figlio** per **ciascuna auto** ed uno per **il semaforo**, e avvalendosi di mutex e condition variable per la sincronizzazione. **Fare in modo che NON RIMANGANO PROCESSI figli ZOMBI mentre il programma è in esecuzione.**

Scrivere il Makefile per compilare e linkare i sorgenti. Inserire il necessario controllo di errore. In caso di errore grave, terminare il programma producendo un avviso a video.

Consiglio: Non chiamate auto la vostre funzioni perché esiste una parola chiave del linguaggio C che si chiama così . Meglio chiamare la vostra funzione Auto (con la A maiuscola) .

Esercizio Esame Pratica - **134 - accumula.sh**

Scrivere due script bash, **accumula.sh** e **contacar.sh,** che svolgono i seguenti compiti:

Lo script **contacar.sh** prende due argomenti, il percorso di un file di input da leggere ed il percorso di un file da creare e in cui mettere l'output. Lo script **contacar.sh** deve leggere il contenuto del file di input e contare il numero di caratteri delle sole righe che contengono la parola "typedef". Lo script **contacar.sh** deve mettere questo numero nel file di output passato come secondo argomento. Se il file di output esiste già allora tale file deve essere sovrascritto in modo che alla fine contenga solo l'output prodotto dallo script contacar.sh

lo script **accumula.sh** lancia per tre volte l'esecuzione in background dello script **contacar.sh** passandogli queste coppie di argomenti:

/usr/include/term_entry.h 1.txt
/usr/include/term.h 2.txt
/usr/include/thread_db.h 3.txt

Dopo avere lanciato i 3 script in background, lo script **accumula.sh** aspetta la fine dei tre script lanciati in background. Dopo che tutti e tre sono terminati, lo script **accumula.sh** usa i tre file 1.txt 2.txt e 3.txt e somma i tre numeri contenuti in quei file.

Infine, lo script **accumula.sh** stampa a video la somma dei 3 numeri letti.